



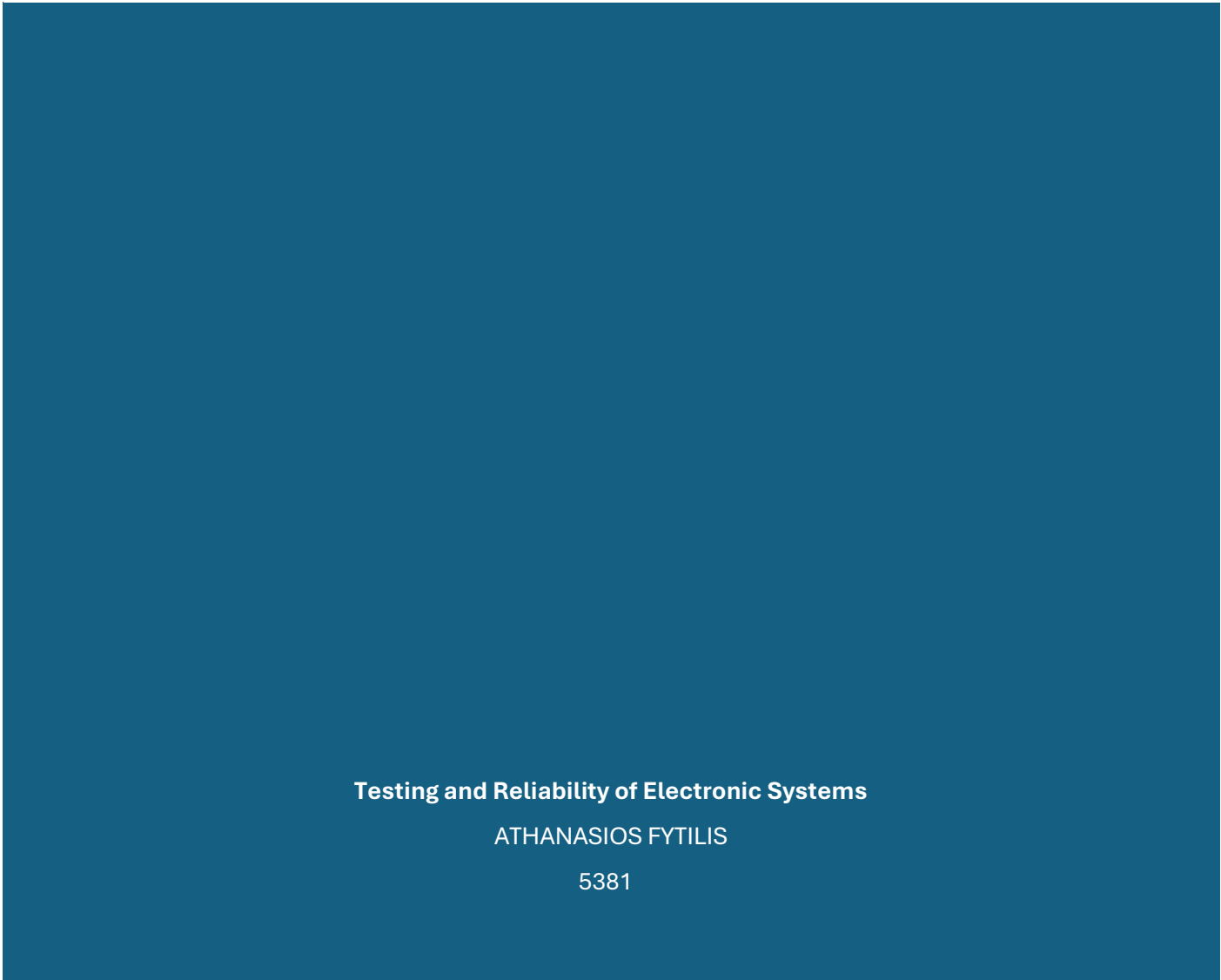
EXERCISE 3

TESTING AND RELIABILITY OF ELECTRONIC SYSTEMS

Testing and Reliability of Electronic Systems

ATHANASIOS FYTILIS

5381



Professor's remark

In exercise 3 the following issues were pointed out to me:

BSC_tb Exercise 3.1

- The transfer of data from the CAP flip-flop to the UPD flip-flop was not clearly visible, because I had not added the Q_UPDATE signal (but I let it be shown later through the testmode), so that the data transfer from the CAP to the UPDATE could be seen immediately.
- I was also correctly suggested to follow a clearer course in this testbench (I had function D first, then A etc.) and not the functions in the order in which the exercise required them.

How did I correct the above remarks?

- I originally added the Q_UPDATE signal so that the reader can have a clear picture when transferring data from the flip-flop CAP to the flip-flop UPD and not have to wait until testmode to see it happen.
- I changed the order of operations in BSC_tb and now they are performed in the order given by the exercise.
- For example, in test mode the DataIn is '0' and the Update from a previous data transfer stays at '1', so that it can be clearly seen where the value comes from in the Data_Out. Something I had before, however I added some wait so that it is even more obvious and not confusing.

3.1 Implementation of JTAG Building Blocks

Introduction

This section describes the design and implementation of the basic structural circuits that make up the **IEEE 1149.1 protocol (JTAG)** in VHDL hardware description language. In order to be able to lead to the final composition of the system, it is necessary to first analyze its individual components separately, starting from the **Boundary-Scan Cell (BSC)** and reaching the architecture of the **TAP Controller**.

More specifically, the following subchapters present:

- **The RTL description and logical design** of the Scan Cell (BSC), analyzing the role of internal Capture and Update registers.
- **The structure of the Finite State Machine (FSM)** that implements the TAP Controller, controlling the navigation in the 16 states of the protocol.
- **The simulations and waveforms in ModelSim**, in which all the necessary control signals were incorporated for the complete verification of the proper functioning of the modules.

A. Bypass Register (BR)

Introduction

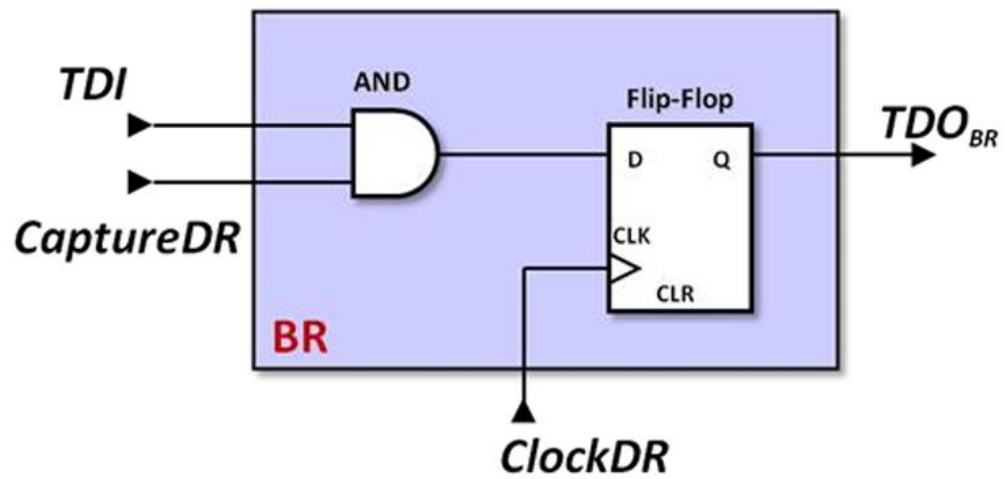
A Bypass Register is a 1-bit shift register. Its operation is critical to system efficiency, as it allows a device that is not participating in the current test to be "bypassed." When activated, it connects the TDI to the TDO through a single delay cycle.

Description of the circuit

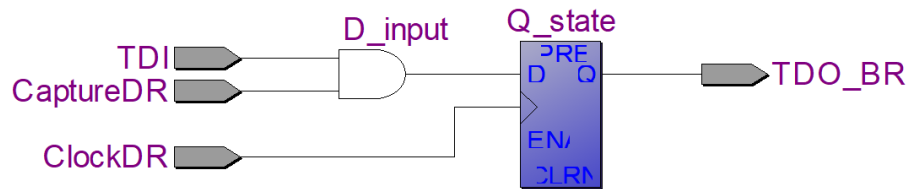
In the BR circuit, we have an AND gate and a Flip-Flop.

- The AND Gate (TDI and CaptureDR):
 - Its role: When the TAP Controller says 'Capture', this gateway forces the Flip-Flop to load the value '0' (according to the JTAG standard).
 - Why? So that the user can know that the connection is working. If we send data and the first thing that comes out is a '0', we know that the chip is responding.
- The Flip-Flop (FF):
 - Its role: It is a 1-bit memory. It keeps the value coming from the TDI for a clock cycle and then lets it go out on the TDO.
 - Why? It creates a controlled latency so that the data does not "run" uncontrollably within the board, but moves step by step with the clock.

Power Point



RTL Viewer



*We observe that schematically our code creates the same circuit as that of the pronunciation and thus we confirm (at least visually) the correctness of our code at the first level.

Code (APPENDIX K1)

Code Description (BR.vhd)

The implementation is based on an AND Gateway and a D Flip-Flop (DFF).

- The input signal of the DFF (D_input) is derived from the AND between the TDI and the CaptureDR control signal.
- The value is stored on the positive edge of the ClockDR clock.
- The output TDO_BR provides the value of the Flip-Flop in the scan chain.

Role in Final Implementation

- BR reduces the length of the overall scanning chain on a PCB with multiple integrated. If we want to check only the 10th chip of a series, the remaining 9 are set to "Bypass" so that the test data reaches its target quickly without going through all the peripheral scan registers.

B. Instruction Register Cell (IR Cell)

Introduction

The command register cell is responsible for storing the test commands (e.g., EXTEST, SAMPLE). It consists of two stages: a Shift Register for serial data entry and a Latch/Update Register for command stabilization.

Testing and Reliability of Electronic Systems

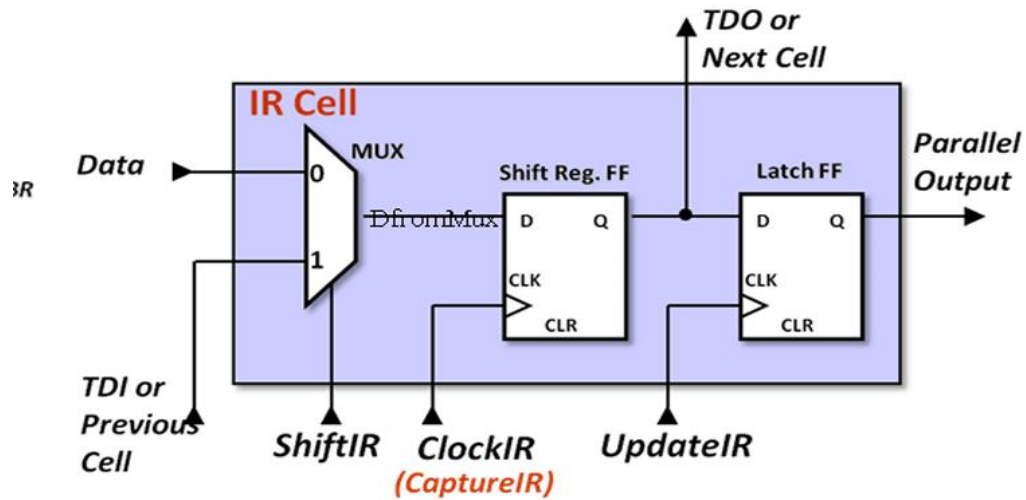
Athanasios Fytilis 5381

Description of the circuit

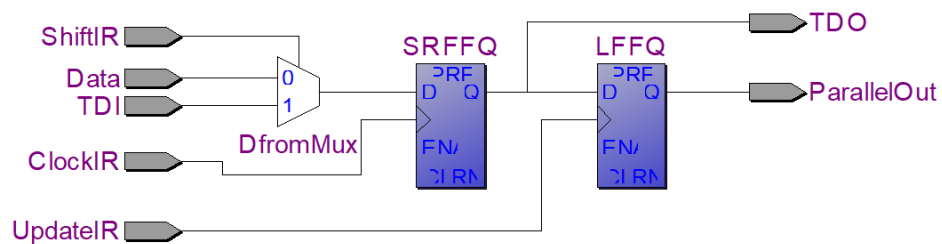
Here we have a **Multiplexer (MUX)** and **two Flip-Flops**.

- **The Multiplexer (MUX):**
 - **Its role:** It is the "traffic warden". It chooses whether the first Flip-Flop will get data from TDI (to load a new command) or from Data (to see the current state of the chip).
- **The Shift Register FF (SRFF):**
 - **Its role:** It is the "conveyor". This FF participates in the shift chain. It accepts the bit, holds it for a while and gives it to the next one.
- **The Latch FF (UPD FF):**
 - **Its role:** It is the "guardian". This FF **does not** participate in the shift. It just waits for the UpdateIR signal. Once it receives it, it copies the value from the SRFF and holds it stable at the output.
 - **Why?** So that the chip command does not change while we "shift" the bits through the SRFF. The chip sees the new command only when we press "Enter" (Update).

Power Point



RTL Viewer



*We observe that schematically our code creates the same circuit as that of the pronunciation and thus we confirm (at least visually) the correctness of our code at the first level.

Code (APPENDIX K2)

Code Description (IR.vhd)

Analysis of Signals (Ports)

To understand the code, we need to know what each "cable" does:

Testing and Reliability of Electronic Systems

Athanasios Fytilis 5381

- **Data:** The value we want to "capture" from the chip.
- **TDI (Test Data In):** The bit that comes from the board's pin or previous cell.
- **ShiftIR:** The control signal. When it's '1', it selects TDI. When it's '0', it selects Data.
- **ClockIR:** The clock that synchronizes the slide.
- **UpdateIR:** The signal that says "Now lock the command you just loaded."
- **ParallelOut:** The final command that goes to the logic of the chip.
- **TDO:** The output that goes to the next cell.

A. The Selector

- If the TAP Controller is in **the Shift-IR state**, ShiftIR becomes '1' and the cell accepts the next bit from the TDI.
- If it is in **the Capture-IR state**, ShiftIR is '0' and the cell loads the internal value Data.

B. The Shift Register FF

This is where the first Flip-Flop (SRFF) comes into play. With each pulse of the clock, it "swallows" the value chosen by the multiplexer above. This FF is responsible for serially transferring the information to the TDO.

C. The Latch/Update FF

This is the most crucial point. The command moved to SRFF is not immediately applied to the chip. The user needs to pulse the UpdateIR (in the Update-IR state). Only then does the value pass to the second Flip-Flop (LFF) and remain constant at the ParallelOut output.

How does it help in the final implementation?

Without the double layer (SRFF and LFF), if we tried to load a 4 bit command, the chip would "think" that we were giving it 4 different commands during the shift! The IR Cell ensures that the chip logic sees the new command **only when the whole sequence is loaded correctly**.

Boundary Scan Cell (BSC)

Introduction

The BSC is a "smart switch" that is placed between the physical pins of the integrated and its internal logic. In normal mode, it is "invisible". But, during testing, it can "cut" the connection and allow the user to:

- Either observe what signals go in or out (Capture).
- Or impose its own values on the pins, bypassing the internal logic (Test/Update).

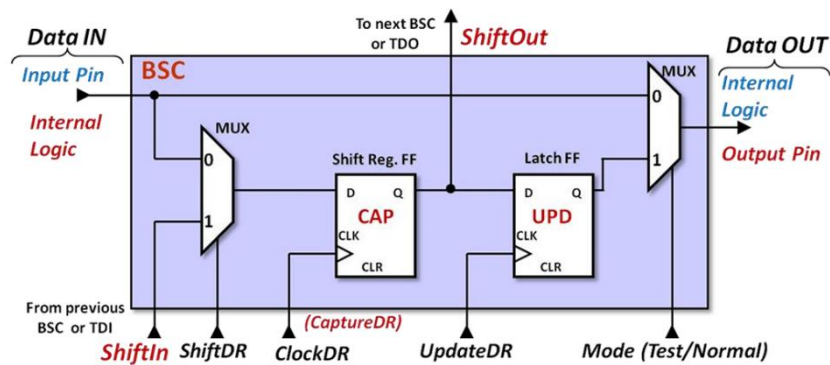
It is the building block that makes it possible to test a board without the use of physical probes, acting as a "**virtual pin**".

Description of the circuit

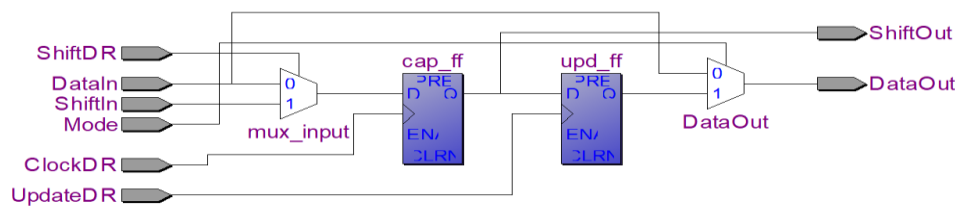
It is the most complex cell, with **two MUX** and **two Flip-Flops (CAP and UPD)**.

- **The first Multiplexer (Input MUX):**
 - It chooses whether to "eavesdrop" the chip pin (DataIn) or whether to accept data from the previous cell (ShiftIn).
- **Capture FF (CAP):**
 - **Its role:** It is the state machine. In Capture mode, it stores the value that passes through the chip pin at that moment. Then, in Shift mode, it sends that photo out for us to check.
- **The FF Update (UPD):**
 - **Its role:** It is the "virtual hand". It stores the value we want to impose on the chip.
- **The second Multiplexer (Output MUX):**
 - **Its role:** It is the "power switch".
 - If Mode = '0', it allows the signal to pass normally (Normal).
 - If Mode = '1', it cuts the connection to the outside world and forces the Pin to take the value stored in **UPD FF**.

Power Point



RTL Viewer



*We observe that schematically our code creates the same circuit as that of the pronunciation and thus we confirm (at least visually) the correctness of our code at the first level.

Code (APPENDIX K3)

Code Description (BSC.vhd)

1. Input Selector:

This multiplexer determines the source of the information.

- When ShiftDR='0', the cell 'looks' at the pin or internal logic (DataIn) to sample.
- When ShiftDR='1', the cell binds to the previous BSC (ShiftIn), creating a long 'Scan Chain' within which the data moves serially.

2. Capture Flip-Flop (CAP FF):

Testing and Reliability of Electronic Systems

Athanasios Fytilis 5381

It is the memory unit of the shift chain. On the positive edge of ClockDR, it stores the value chosen by the first multiplexer. Its output (ShiftOut) is led to the next BSC. So, we can "photograph" a situation and then "shift" it out of the chip for control.

3.Update Flip-Flop (UPD FF):

It is an auxiliary retention register. Its job is to keep a constant value for the pin while we shift new data into Capture FF. Without it, the chip outputs would flash uncontrollably during data shifting.

Output Multiplexer (Mode Selector):

4.It is the "central switch" of operation:

- Normal Mode (Mode='0'): The signal passes normally (bypass BSC). The chip works as if JTAG didn't exist.
- Test Mode (Mode='1'): The internal logic is isolated and the pin is driven by the value we have loaded in the Update register.

Role in Final Implementation and Future Use

BSC is the "hand" that allows us to touch the inside of the chip. In the final implementation, hundreds of such cells are connected in series, forming the Boundary Scan Register (BSR).

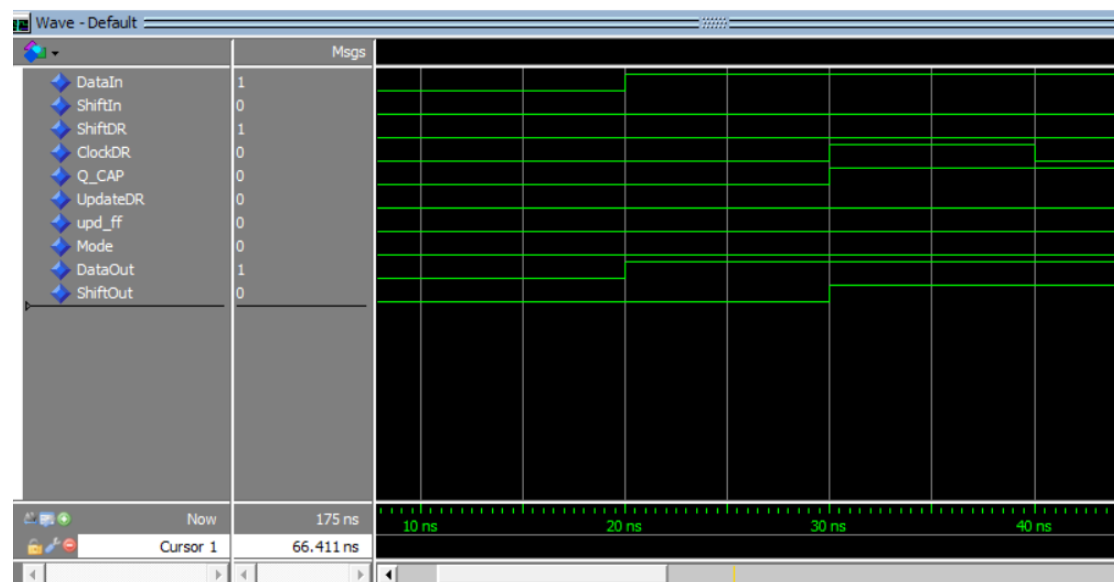
Implementation of Testbench to verify the operation of the BSC cell

CODE (APPENDIX K4)

Description of the code

A) Save to CAP FF by Internal Logic (Capture)

- **What to look out for:** The ShiftDR signal should be '0'.
- **Critical Point:** At the rising edge of ClockDR, the input multiplexer chooses DataIn.
- **Success:** The internal signal cap_ff (Q_CAP) and the ShiftOut output should show the value of DataIn ('1' in our case).



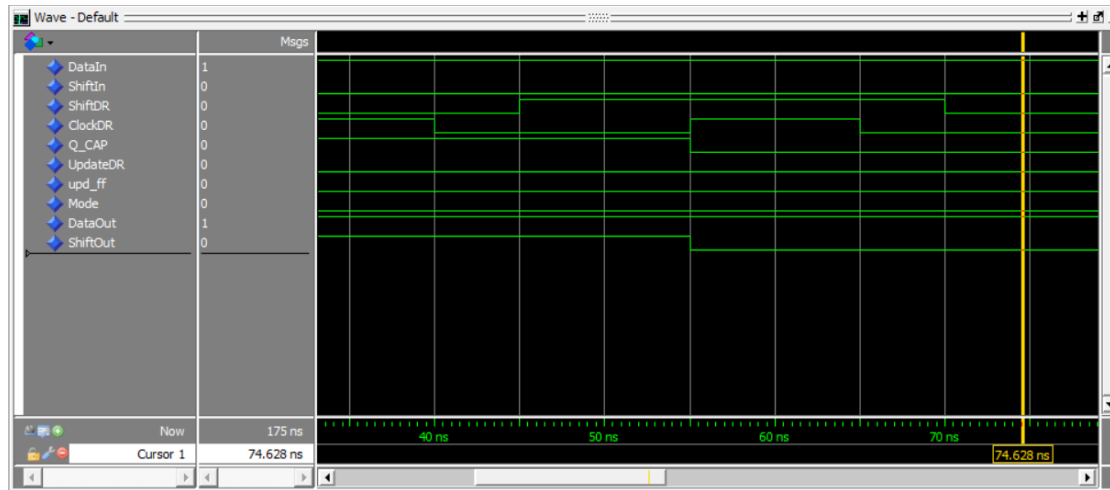
And indeed, looking at the snapshot of our simulation, the correct functioning of A is verified.

B) Save to flip-flop CAP with data from shiftin

- **What we look out for:** The ShiftDR signal should be '1'.
- **Critical point:** The cell now functions as a shift register. With ClockDR, it samples ShiftIn (TDI).
- **Success:** cap_ff (Q_CAP) is updated with the new value (of Shift In), ignoring DataIn.

Testing and Reliability of Electronic Systems

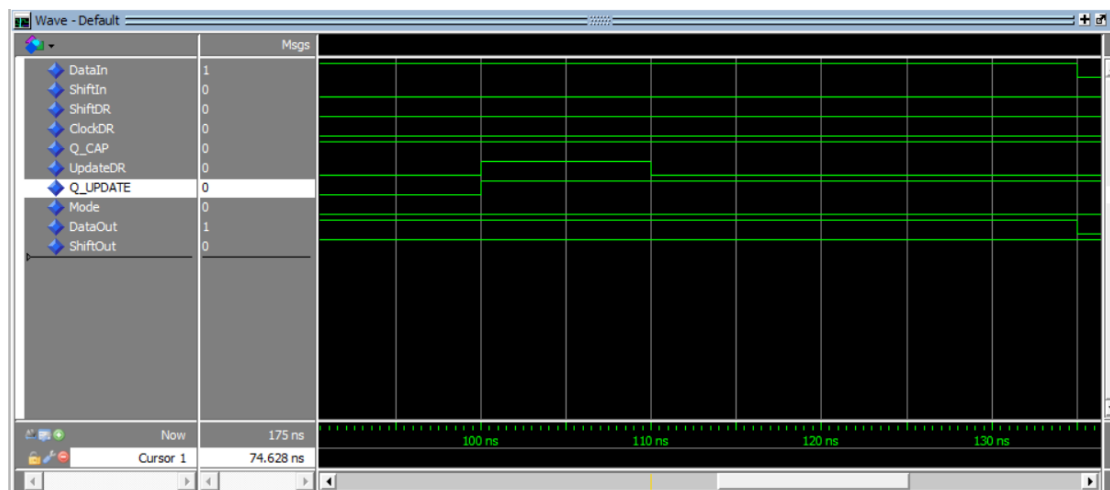
Athanasios Fytillis 5381



And indeed, looking at the snapshot of our simulation verifies the correct functioning of B.

C) data transfer from flip-flop CAP to flip-flop UPD

- **What we're looking out for:** The rising edge of the UpdateDR signal.
- **Critical point:** The value that was "locked" in CAP FF is transferred to the upd_ff register (Q_UPDT).
- **Success:** In the simulation, the upd_ff(Q_UPDATE) signal must be matched with cap_ff(Q_CAP). This value will be used in the next stage (Test Mode).



And indeed, looking at the snapshot of our simulation verifies the correct functioning of C.

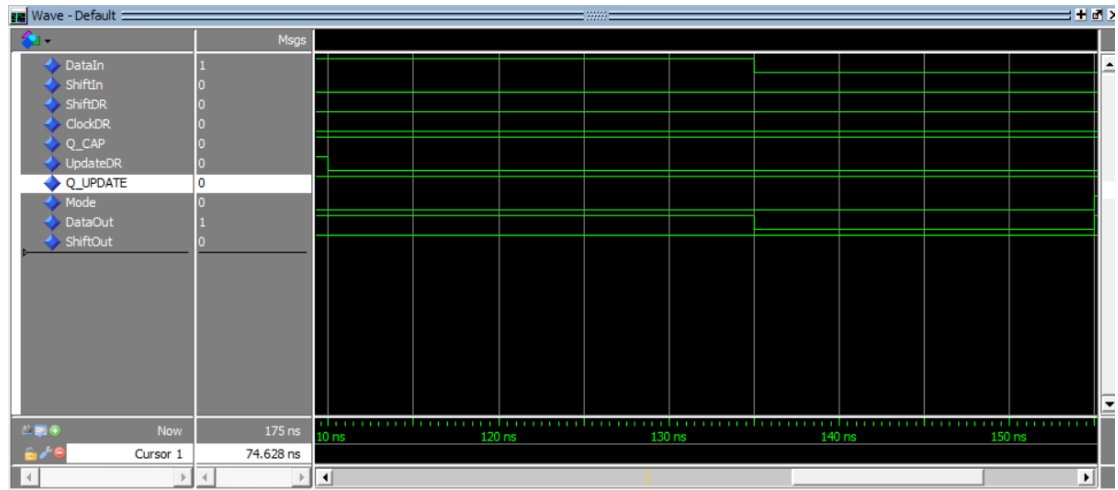
D) normal mode: forwarding data from internal logic to internal logic

- **What to watch out for:** The Mode signal should be '0'.

Testing and Reliability of Electronic Systems

Athanasios Fytilis 5381

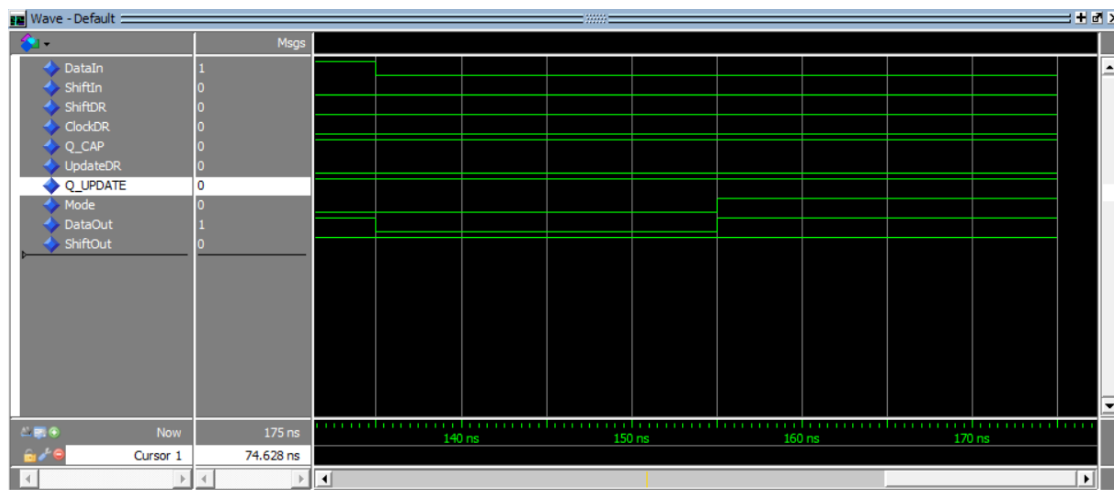
- **Critical point:** The output multiplexer directly connects the DataIn to the DataOut.
- **Success:** The circuit is "transparent". Every change in DataIn is immediately reflected in DataOut without clock lag



And indeed, looking at the waveform snapshot confirms the proper functioning of normal mode.

E) test mode: forwarding data from UPD flip-flop to internal logic

- **What to look out for:** The Mode signal should be '1'.
- **Critical point:** The DataOut output should now ignore the DataIn and only show what is stored in the Update register (upd_ff).
- **Success:** If DataIn is '0' but the DataOut output remains '1' (because we had Update '1' in the previous step), then the cell successfully isolates the internal circuit and performs the Test correctly.



And indeed, looking at the waveform snapshot confirms the correct operation of the test mode.

3.2 Implementation of the TAP Controller (Test Access Port)

1. Introduction

The TAP Controller is a 16-state serial Finite State Machine (FSM), which is the heart of the JTAG protocol. Its role is to interpret the **TMS** (Test Mode Select) control signal on the edge of the **TCK** clock and guide the chip to the appropriate test modes.

The situations are divided into two main paths:

- **Data Path (DR):** To handle the states of data registrars (e.g., BSC, Bypass).
- **Instruction Path (IR):** To load new commands into the Instruction Register.

A critical feature of the TAP Controller is that, regardless of the current state, if the **TMS** signal remains high ('1') for 5 consecutive clock cycles, the state machine always safely returns to the original **Test-Logic-Reset state**.

Description of the circuit

1. FSM Block & Feedback

The center of the circuit is dominated by the `current_s` unit. Although in RTL Viewer it appears as a single functional block, internally it includes the **registers that** store the current state.

- **The Bus:** The thick line that exits the block and returns to the input of the combinatorial logic is the **Feedback Loop**. This transfers the encoded information of the current state back to the "Next State" logic.
- **The Decision Logic:** Before the central block, a cluster of gateways receives the **TMS** signal and, combined with the current state from the feedback, calculates what the next step will be in the state diagram.

2. Control Inputs

- **TCK Signal:** The system clock connects directly to the timing input of the state block, ensuring that transitions are made synchronously at its positive edge.
- **TRST Signal:** This signal binds to the asynchronous zeroing inputs of the internal Flip-Flops, allowing the controller to be immediately reset to the **Test-Logic-Reset state**.

3. Output Decoding Unit

In the right part of the RTL Viewer we observe two distinct exit paths:

- **state_bin:** A simple combinatorial decoder that converts the internal state into the requested 4-bit binary code.

Testing and Reliability of Electronic Systems

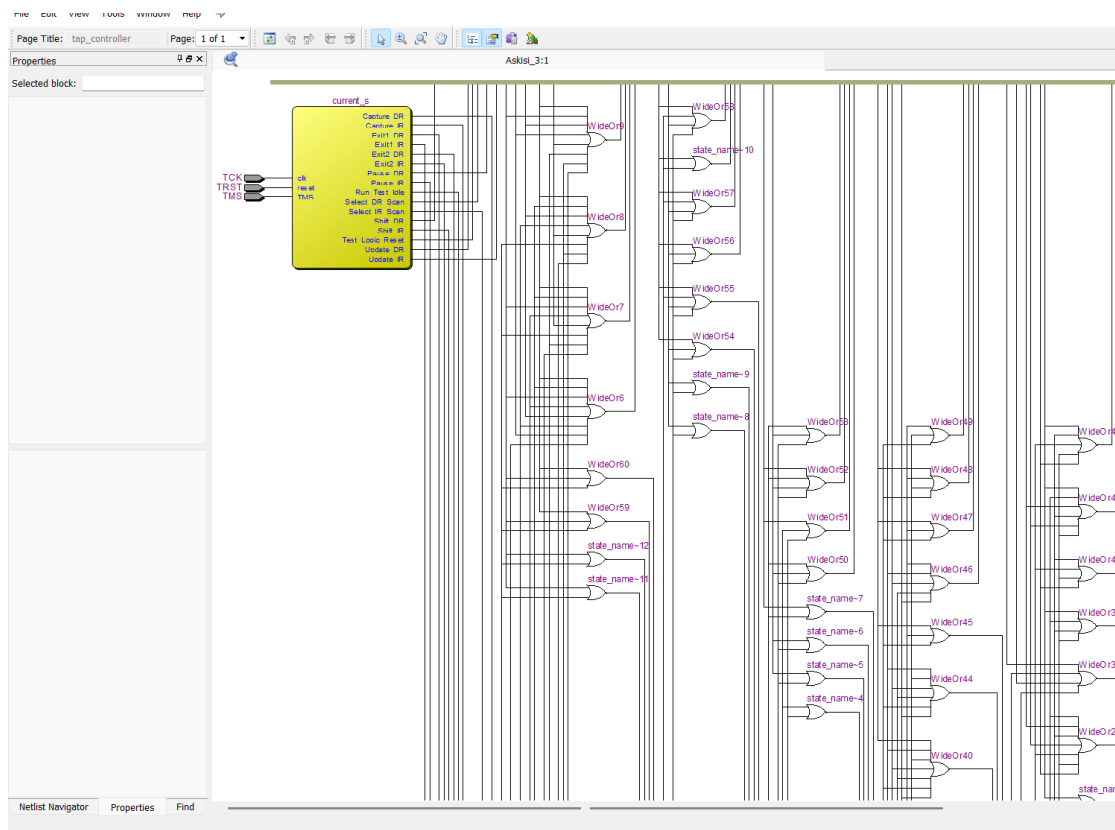
Athanasios Fytilis 5381

- **state_name:** This is the most extensive section of RTL, consisting of a series of large **multiplexers**. These multiplexers choose between fixed ASCII values of characters (hardwired constants) to form the 16-character String that corresponds to each state.

Power Point

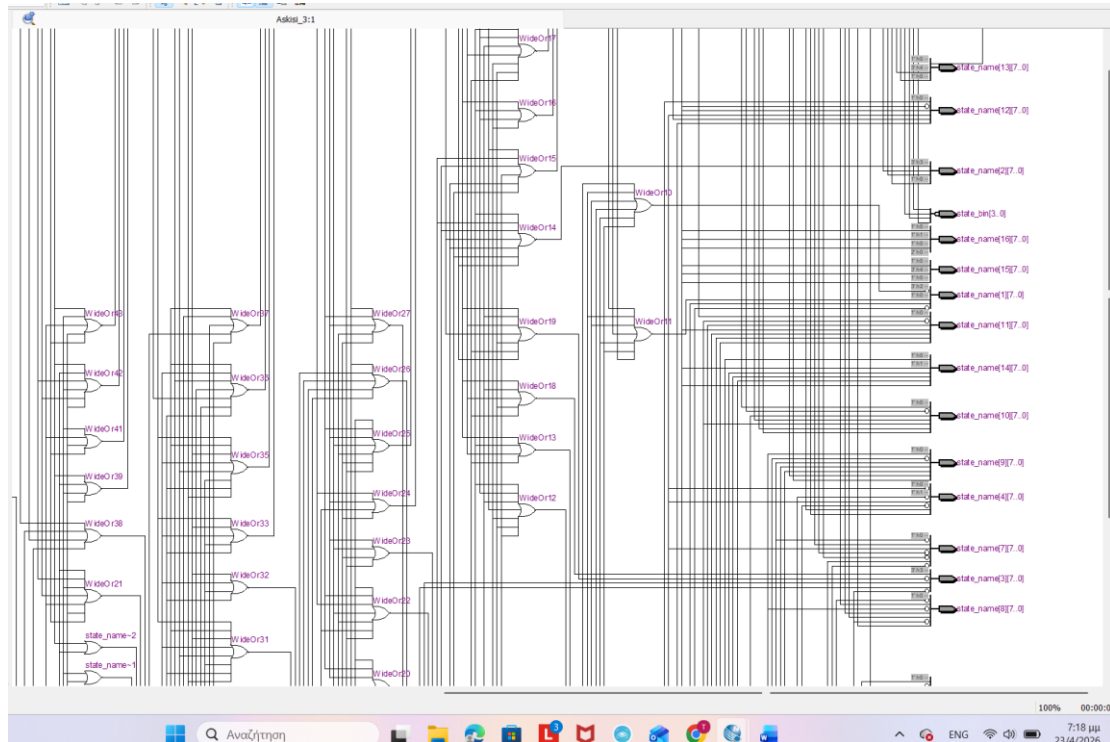


RTL Viewer



Testing and Reliability of Electronic Systems

Athanasios Fytillis 5381



Code (tap_controller.vhd) APPENDIX K5

2. Detailed Code Description (tap_controller.vhd)

The implementation in VHDL follows the FSM standard with two processes for maximum stability:

1. **Enumerated Type:** Instead of simple numbers, we used a state_type formula that includes the names of all 16 states (e.g., Capture_DR, Shift_IR). This makes the code readable and reduces errors.

2. **Sequential Process:**

```
if TRST = '1' then current_s <= Test_Logic_Reset;  
elsif rising_edge(TCK) then current_s <= next_s;
```

This process is responsible for changing the state. It uses **TRST** as an asynchronous reset to instantly initialize the controller.

3. **Combinatorial Next-State Logic:** A case command is used that checks the current state (current_s). In each case, an if-else command looks at the value of **the TMS**. For example, if we are in the Shift_DR and the TMS becomes '1', the machine switches to the Exit1_DR, otherwise it stays in the Shift_DR to continue sliding the bits.
4. **ASCII Encoding for Debugging (State Name Output):** As shown in the code, a STRING-type output has been added. This "translation" of the state

Testing and Reliability of Electronic Systems

Athanasios Fytilis 5381

into text (e.g., "Shift-DR") is crucial for verification through the testbench, as it allows the designer to track the controller's path in the waveform without having to decode binaries.

Code(tb_tap_controller) APPENDIX K6

Code Description

1. Entity & Signals

- **Component Declaration:** The unit to be controlled (tap_controller) is declared with the input (TCK, TMS, TRST) and output (state_bin, state_name) ports.
- **Control Signals:** Local signals (e.g., TCK_sig, TMS_sig) are generated to connect to the entity. Of particular importance is the constant TCK_period (10 ns), which determines the operating speed of the system.

2. Clock Production (TCK Process)

The process TCK_process generates the square timing signal. It switches the TCK_sig value from '0' to '1' every 5 ns (half period), ensuring a consistent pace for the state machine to operate.

3. Test Scenarios (Stimulus Process)

The stim_proc process is the "heart" of the audit and follows the following stages:

- **Reset Path:** The TRST_sig signal is used for 15 ns to enforce the Test-Logic-Reset state. The code then uses the wait until command falling_edge(TCK_sig). This is critical: we change the TMS to the negative edge so that it is perfectly stable when the rising edge comes, thus avoiding race conditions.
- **Data Path (DR Path):** The testbench takes the testbench from Run-Test/Idle to Select-DR-Scan and runs along the entire right side of the diagram. It includes

Testing and Reliability of Electronic Systems

Athanasios Fytilis 5381

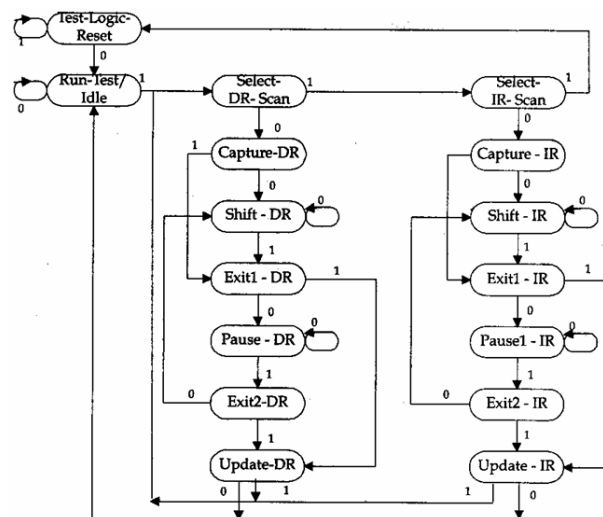
loops in the Shift-DR and Pause-DR modes to simulate multi-bit shifting and process pause.

- **IR Path:** Accordingly, the left side of the diagram is traversed to load commands. The scenario involves the transition from Capture-IR to Update-IR and the eventual return to the resting state.
- **5-Ones Reset: The JTAG protocol's** requirement that with 5 consecutive '1' in the TMS, the controller always returns to the original Test-Logic-Reset state, regardless of where it was located, is verified.

4. Simulation Termination

At the end of the path, the assert false report "End" severity failure? command is used. This automatically stops the simulation in ModelSim once all paths have been successfully completed, preventing unnecessary data from being generated.

FSM



The image displays three sequential screenshots of a logic analyzer interface, showing test sequences and timing diagrams. Each screenshot includes a list of test steps on the left and a corresponding timing diagram on the right.

Top Screenshot:

- Test Steps:**
 - /tb_tap_controller/... 0
 - /tb_tap_controller/... 0
 - /tb_tap_controller/... 0
 - /tb_tap_controller/s... 0000
 - /tb_tap_controller/s... Test-Logic-Reset
- Timing Diagram:** Shows a sequence of signals (0, 1, 0000, 0001, 0010) and their transitions over time. A yellow cursor is positioned at 34.209 ns.

Middle Screenshot:

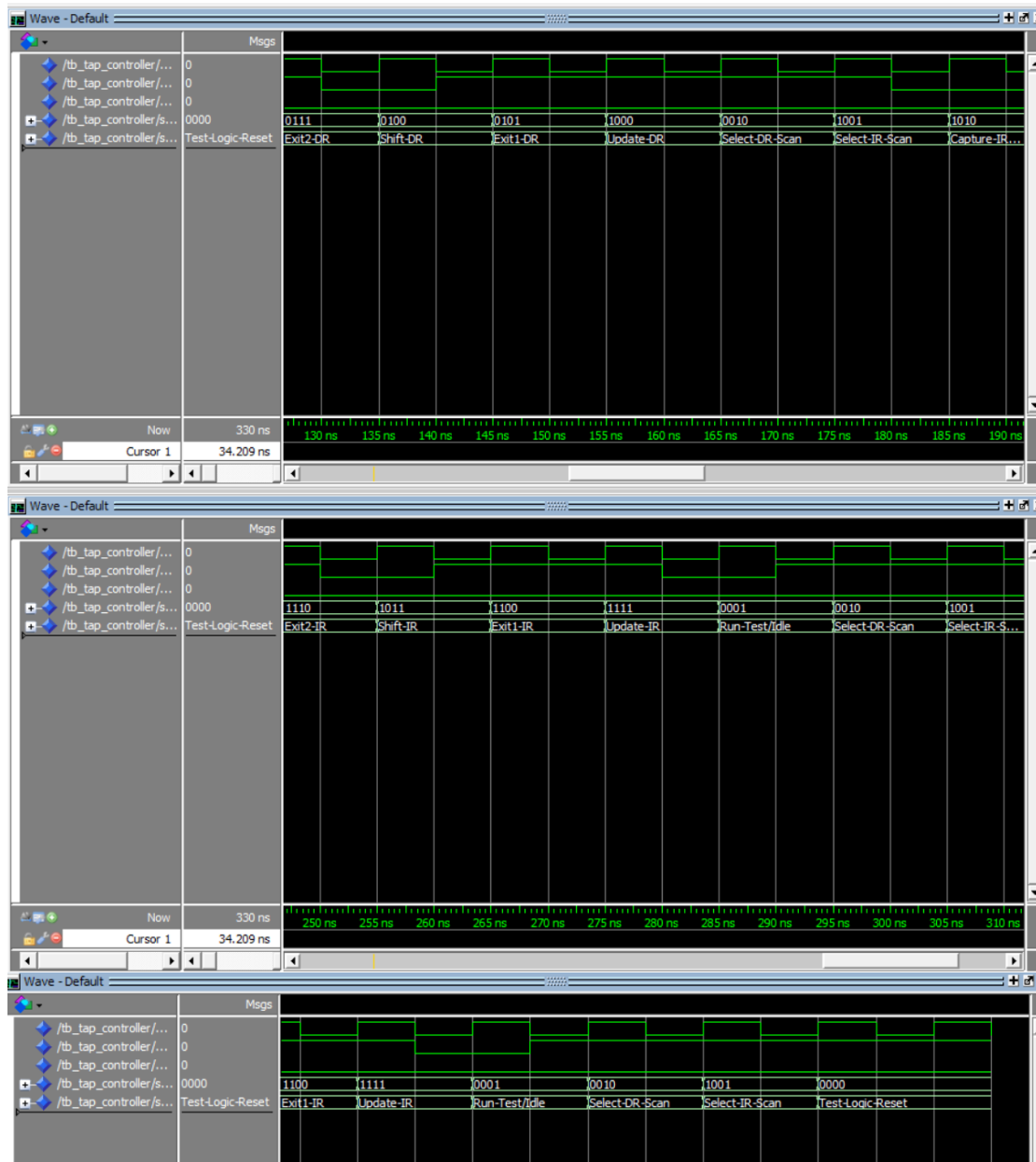
- Test Steps:**
 - /tb_tap_controller/... 0
 - /tb_tap_controller/... 0
 - /tb_tap_controller/... 0
 - /tb_tap_controller/s... 0000
 - /tb_tap_controller/s... Test-Logic-Reset
- Timing Diagram:** Shows a sequence of signals (1010, 1011, 1100, 1101, 1110) and their transitions over time. A yellow cursor is positioned at 34.209 ns.

Bottom Screenshot:

- Test Steps:**
 - /tb_tap_controller/... 0
 - /tb_tap_controller/... 0
 - /tb_tap_controller/... 0
 - /tb_tap_controller/s... 0000
 - /tb_tap_controller/s... Test-Logic-Reset
- Timing Diagram:** Shows a sequence of signals (0010, 0011, 0100, 0101, 0110) and their transitions over time. A yellow cursor is positioned at 34.209 ns.

Testing and Reliability of Electronic Systems

Athanasios Fytillis 5381



1. Reset & Idle Script:

- TRST asynchronous input has been verified.
- Self-loops in Test-Logic-Reset (TMS=1) and Run-Test/Idle (TMS=0) were verified.

2. Data Register Script (DR Scan Chain):

- We ran all the way to the right side of the figure.
- We checked the Shift-DR loop (for multi-bit sliding).
- We checked the Pause-DR route and the return via Exit2-DR back to the skid. This proves that the tester can "freeze" the test and resume it.
- Verified return to the top via Update-DR -> Select-DR.

Testing and Reliability of Electronic Systems

Athanasios Fytilis 5381

3. Instruction Register Script (IR Scan Chain):

- We ran along the left side of the figure.
- Command load (Capture, Shift, Update) verified.
- It has been confirmed that after the Update-IR, the controller can return to rest (Run-Test/Idle).

4. Safety Scenario (Emergency Reset):

- JTAG's critical requirement has been verified: At any point (here from Select-IR), if the TMS remains '1', the engine returns to Test-Logic-Reset.

Conclusion:

The TAP Controller is the one that generates all the control signals we saw in Exercise 3.1 (such as ShiftDR, ClockIR, UpdateDR). Essentially, it "tells" the BSCs and the IR when to Capture, when to transfer data (Shift) and when to apply the values (Update).

APPENDIX

APPENDIX K1 BR.vhd

```
library IEEE;

use IEEE. STD_LOGIC_1164.ALL;


entity BR is

    Port (
        TDI      : in  STD_LOGIC;
        CaptureDR : in  STD_LOGIC;
        ClockDR   : in  STD_LOGIC;
        TDO_BR    : out STD_LOGIC
    );
end BR;


architecture Behavioral of BR is

    signal D_input : STD_LOGIC;
    signal Q_state : STD_LOGIC := '0';
begin

    -- AND Gateway
    D_input <= TDI and CaptureDR;


    -- Flip-Flop
    process(ClockDR)
    begin
        if rising_edge(ClockDR) then
            Q_state <= D_input;
        end if;
    end process;
end Behavioral;
```

Testing and Reliability of Electronic Systems
Athanasios Fytilis 5381

```
    end process;  
TDO_BR <= Q_state;  
end Behavioral;
```

APPENDIX K2 IR.vhd

```
library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;  
  
entity IR is  
    Port (  
        Data      : in STD_LOGIC;  
        TDI       : in STD_LOGIC;  
        ShiftIR   : in STD_LOGIC;  
        ClockIR   : in STD_LOGIC; -- CaptureIR  
        UpdateIR  : in STD_LOGIC;  
        ParallelOut : out STD_LOGIC;  
        TDO       : out STD_LOGIC  
    );  
end IR;  
  
architecture Behavioral of IR is  
    signal SRFFQ : STD_LOGIC := '0';  
    signal LFFQ  : STD_LOGIC := '0';  
    signal DfromMux : STD_LOGIC;  
begin  
    -- Implementation of the Multiplex (Mux)  
    DfromMux <= TDI when ShiftIR = '1' else Data;  
  
    -- Implementation of Shift Register Flip-Flop (SRFF)
```


Testing and Reliability of Electronic Systems

Athanasios Fytilis 5381

```
process(ClockIR)
begin
    if rising_edge(ClockIR) then
        SRFFQ <= DfromMux;
    end if;
end process;

-- Implementation of Latch Flip-Flop (LFF)
process(UpdateIR)
begin
    if rising_edge(UpdateIR) then
        LFFQ <= SRFFQ;
    end if;
end process;

TDO <= SRFFQ;
ParallelOut <= LFFQ;
end Behavioral;
```

APPENDIX K3 BSC.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity bsc is
    Port (
        DataIn    : in STD_LOGIC; -- From Pin or Internal Logic
        ShiftIn    : in STD_LOGIC; -- From previous BSC or TDI
        ShiftDR    : in STD_LOGIC; -- Select Source (0: DataIn, 1: ShiftIn)
```

Testing and Reliability of Electronic Systems

Athanasios Fytilis 5381

```
ClockDR   : in STD_LOGIC; -- Clock for Capture FF
UpdatedR  : in STD_LOGIC; -- Clock for Update FF
Mode      : in STD_LOGIC; -- Mode Selection (0: Normal, 1: Test)
DataOut   : out STD_LOGIC; -- To Internal Logic or Pin
ShiftOut  : out STD_LOGIC -- To Next BSC or TDO

);
end bsc;
```

architecture Behavioral of bsc is

```
    signal cap_ff : STD_LOGIC := '0';
    signal upd_ff : STD_LOGIC := '0';
    signal mux_input : STD_LOGIC;

begin

    -- Input multiplexer
    mux_input <= ShiftIn when ShiftDR = '1' else DataIn;

    -- Capture Flip-Flop (CAP)
    process(ClockDR) begin
        if rising_edge(ClockDR) then
            cap_ff <= mux_input;
        end if;
    end process;

    -- Update Flip-Flop (UPD)
    process(UpdateDR) begin
        if rising_edge(UpdateDR) then
            upd_ff <= cap_ff;
        end if;
    end process;
```

Testing and Reliability of Electronic Systems
Athanasios Fytilis 5381

end process;

-- Output Multiplexer (Mode Control)

DataOut <= upd_ff when Mode = '1' else DataIn;

-- ShiftOut output is always the output of CAP FF

ShiftOut <= cap_ff;

end Behavioral;

APPENDIX K4 tb_bsc.vhd

library IEEE;

use IEEE. STD_LOGIC_1164.ALL;

entity tb_bsc is

end tb_bsc;

architecture behavior of tb_bsc is

component bsc

Port (

DataIn, ShiftIn, ShiftDR, ClockDR, UpdateDR, Mode : in STD_LOGIC;

DataOut, ShiftOut : out STD_LOGIC

);

end component;

signal DataIn, ShiftIn, ShiftDR, ClockDR, UpdateDR, Mode : STD_LOGIC := '0';

signal DataOut, ShiftOut : STD_LOGIC;

begin

Testing and Reliability of Electronic Systems

Athanasios Fytilis 5381

uut: bsc Port Map (DataIn, ShiftIn, ShiftDR, ClockDR, UpdateDR, Mode, DataOut, ShiftOut);

stim_proc: process

begin

wait for 20 ns;

-- A) Save to CAP FF by Internal Logic (Capture)

ShiftDR <= '0';

 DataIn <= '1';

 wait for 10 ns;

ClockDR <= '1';

 wait for 10 ns; ClockDR <= '0';

-- VERIFY: cap_ff should be made '1'

wait for 5 ns;

-- B) Save to CAP FF by ShiftIn (Shift)

ShiftDR <= '1'; ShiftIn <= '0'; wait for 10 ns;

ClockDR <= '1'; wait for 10 ns; ClockDR <= '0';

-- VERIFY: cap_ff should be made '0'

wait for 5 ns;

-- C) Transfer from CAP FF to UPD FF (Update)

Testing and Reliability of Electronic Systems

Athanasios Fytilis 5381

-- Preparation: Put '1' on the CAP FF

ShiftDR <= '0'; DataIn <= '1'; wait for 10 ns;

ClockDR <= '1'; wait for 10 ns; ClockDR <= '0';

-- Run Update

wait for 10 ns;

UpdateDR <= '1'; wait for 10 ns; UpdateDR <= '0';

-- VERIFY: upd_ff gets the value '1'

wait for 5 ns;

-- D) Normal Mode: DataIn Forwarding -> DataOut

Mode <= '0'; DataIn <= '1'; wait for 20 ns;

-- VERIFY: DataOut = DataIn immediately

DataIn <= '0'; wait for 20 ns;

-- E) Test Mode: Forward UPD FF -> DataOut (Isolation)

Mode <= '1'; -- Enable Test Mode [cite: 85]

DataIn <= '0'; -- Change DataIn for contrast control

wait for 20 ns;

-- VERIFY: DataOut = '1' (from UPD) and NOT '0' (from DataIn)

report "BSC Verification Completed!" severity note;

assert false report "End of Simulation" severity failure;

Testing and Reliability of Electronic Systems
Athanasios Fytilis 5381

```
end process;  
end behavior;
```

APPENDIX K5 tap_controller.vhd

```
library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;  
  
entity tap_controller is  
    Port (  
        TCK    : in STD_LOGIC;  
        TMS    : in STD_LOGIC;  
        TRST   : in STD_LOGIC;  
        state_bin : out STD_LOGIC_VECTOR(3 downto 0); -- Binary output (state)  
        state_name : out STRING(1 to 16) -- ASCII output for debugging  
    );  
end tap_controller;  
  
architecture Behavioral of tap_controller is  
    -- Definition of 16 states
```

Testing and Reliability of Electronic Systems
Athanasios Fytilis 5381

```
type state_type is (
    Test_Logic_Reset, Run_Test_Idle,
    Select_DR_Scan, Capture_DR, Shift_DR, Exit1_DR, Pause_DR, Exit2_DR,
    Update_DR,
    Select_IR_Scan, Capture_IR, Shift_IR, Exit1_IR, Pause_IR, Exit2_IR,
    Update_IR
);
signal current_s, next_s : state_type;

begin
    -- Modern transition of situations
    process(TCK, TRST)
    begin
        if TRST = '1' then
            current_s <= Test_Logic_Reset; -- Asynchronous Reset
        elsif rising_edge(TCK) then
            current_s <= next_s;
        end if;
    end process;

    -- TMS-based next-state logic
    process(current_s, TMS)
    begin
        case current_s is
            when Test_Logic_Reset => if TMS = '0' then next_s <= Run_Test_Idle; else
next_s <= Test_Logic_Reset; end if;
            when Run_Test_Idle => if TMS = '1' then next_s <= Select_DR_Scan; else
next_s <= Run_Test_Idle; end if;

            -- Data Register Path (DR)
```

Testing and Reliability of Electronic Systems

Athanasios Fytilis 5381

```
when Select_DR_Scan => if TMS = '0' then next_s <= Capture_DR; else  
next_s <= Select_IR_Scan; end if;
```

```
when Capture_DR => if TMS = '0' then next_s <= Shift_DR; else next_s <=  
Exit1_DR; end if;
```

```
when Shift_DR => if TMS = '0' then next_s <= Shift_DR; else next_s <=  
Exit1_DR; end if;
```

```
when Exit1_DR => if TMS = '0' then next_s <= Pause_DR; else next_s <=  
Update_DR; end if;
```

```
when Pause_DR => if TMS = '0' then next_s <= Pause_DR; else next_s <=  
Exit2_DR; end if;
```

```
when Exit2_DR => if TMS = '0' then next_s <= Shift_DR; else next_s <=  
Update_DR; end if;
```

```
when Update_DR => if TMS = '0' then next_s <= Run_Test_Idle; else  
next_s <= Select_DR_Scan; end if;
```

-- Instruction Register (IR) Path

```
when Select_IR_Scan => if TMS = '0' then next_s <= Capture_IR; else  
next_s <= Test_Logic_Reset; end if;
```

```
when Capture_IR => if TMS = '0' then next_s <= Shift_IR; else next_s <=  
Exit1_IR; end if;
```

```
when Shift_IR => if TMS = '0' then next_s <= Shift_IR; else next_s <=  
Exit1_IR; end if;
```

```
when Exit1_IR => if TMS = '0' then next_s <= Pause_IR; else next_s <=  
Update_IR; end if;
```

```
when Pause_IR => if TMS = '0' then next_s <= Pause_IR; else next_s <=  
Exit2_IR; end if;
```

```
when Exit2_IR => if TMS = '0' then next_s <= Shift_IR; else next_s <=  
Update_IR; end if;
```

```
when Update_IR => if TMS = '0' then next_s <= Run_Test_Idle; else  
next_s <= Select_DR_Scan; end if;
```

```
when others => next_s <= Test_Logic_Reset;
```


Testing and Reliability of Electronic Systems
Athanasios Fytilis 5381

```
end case;

end process;

-- Convert state to Binary and ASCII (String)

process(current_s)

begin

    case current_s is

        when Test_Logic_Reset => state_bin <= "0000"; state_name <= "Test-Logic-Reset";

        when Run_Test_Idle  => state_bin <= "0001"; state_name <= "Run-Test/Idle ";

        when Select_DR_Scan => state_bin <= "0010"; state_name <= "Select-DR-Scan ";

        when Capture_DR    => state_bin <= "0011"; state_name <= "Capture-DR ";

        when Shift_DR      => state_bin <= "0100"; state_name <= "Shift-DR    ";

        when Exit1_DR      => state_bin <= "0101"; state_name <= "Exit1-DR    ";

        when Pause_DR     => state_bin <= "0110"; state_name <= "Pause-DR   ";

        when Exit2_DR     => state_bin <= "0111"; state_name <= "Exit2-DR    ";

        when Update_DR    => state_bin <= "1000"; state_name <= "Update-DR   ";

        when Select_IR_Scan => state_bin <= "1001"; state_name <= "Select-IR-Scan ";

        when Capture_IR   => state_bin <= "1010"; state_name <= "Capture-IR  ";

        when Shift_IR     => state_bin <= "1011"; state_name <= "Shift-IR    ";

        when Exit1_IR     => state_bin <= "1100"; state_name <= "Exit1-IR    ";

        when Pause_IR    => state_bin <= "1101"; state_name <= "Pause-IR    ";

        when Exit2_IR    => state_bin <= "1110"; state_name <= "Exit2-IR    ";

    end case;

end process;
```

Testing and Reliability of Electronic Systems
Athanasios Fytilis 5381

```
        when Update_IR    => state_bin <= "1111"; state_name <= "Update-IR    ";  
    end case;  
end process;  
end Behavioral;
```

APPENDIX K6 tb_tap_controller.vhd

```
library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;  
  
entity tb_tap_controller is  
end tb_tap_controller;  
  
architecture behavior of tb_tap_controller is  
    component tap_controller  
        Port (  
            TCK      : in STD_LOGIC;  
            TMS      : in STD_LOGIC;  
            TRST     : in STD_LOGIC;  
            state_bin : out STD_LOGIC_VECTOR(3 downto 0);  
            state_name : out STRING(1 to 16)  
        );  
    end component;  
  
    signal TCK_sig : STD_LOGIC := '0';  
    signal TMS_sig : STD_LOGIC := '1'; -- We start with 1 for the Reset  
    signal TRST_sig : STD_LOGIC := '0';  
    signal state_sig : STD_LOGIC_VECTOR(3 downto 0);  
    signal state_name_sig : STRING(1 to 16);
```

Testing and Reliability of Electronic Systems

Athanasios Fytilis 5381

```
constant TCK_period : time := 10 ns;

begin

    uut: tap_controller Port Map (TCK_sig, TMS_sig, TRST_sig, state_sig,
state_name_sig);

    -- Clock Generation

    TCK_process : process
    begin
        TCK_sig <= '0'; wait for TCK_period/2;
        TCK_sig <= '1'; wait for TCK_period/2;
    end process;

    stim_proc: process
    begin
        -- 1. Initialization & Reset Loop
        TRST_sig <= '1'; wait for 15 ns; TRST_sig <= '0';
        wait until falling_edge(TCK_sig);
        TMS_sig <= '1'; wait until falling_edge(TCK_sig); -- Stay in Test-Logic-Reset
        TMS_sig <= '0'; wait until falling_edge(TCK_sig); -- To Run-Test/Idle
        TMS_sig <= '0'; wait until falling_edge(TCK_sig); -- Stay in Run-Test/Idle

        -- 2. Full Data Register Path (DR Path) with Loops
        TMS_sig <= '1'; wait until falling_edge(TCK_sig); -- To Select-DR-Scan
        TMS_sig <= '0'; wait until falling_edge(TCK_sig); -- To Capture-DR
        TMS_sig <= '0'; wait until falling_edge(TCK_sig); -- To Shift-DR
        TMS_sig <= '0'; wait until falling_edge(TCK_sig); -- Stay in Shift-DR (Loop)
        TMS_sig <= '1'; wait until falling_edge(TCK_sig); -- To Exit1-DR
```

Testing and Reliability of Electronic Systems

Athanasios Fytilis 5381

```
TMS_sig <= '0'; wait until falling_edge(TCK_sig); -- To Pause-DR
TMS_sig <= '0'; wait until falling_edge(TCK_sig); -- Stay in Pause-DR (Loop)
TMS_sig <= '1'; wait until falling_edge(TCK_sig); -- To Exit2-DR
TMS_sig <= '0'; wait until falling_edge(TCK_sig); -- Back to Shift-DR
TMS_sig <= '1'; wait until falling_edge(TCK_sig); -- To Exit1-DR
TMS_sig <= '1'; wait until falling_edge(TCK_sig); -- To Update-DR
TMS_sig <= '1'; wait until falling_edge(TCK_sig); -- To Select-DR-Scan
(Loopback)
```

-- 3. Full Instruction Register (IR Path) with Loops

```
TMS_sig <= '1'; wait until falling_edge(TCK_sig); -- To Select-IR-Scan
TMS_sig <= '0'; wait until falling_edge(TCK_sig); -- To Capture-IR
TMS_sig <= '0'; wait until falling_edge(TCK_sig); -- To Shift-IR
TMS_sig <= '0'; wait until falling_edge(TCK_sig); -- Stay in Shift-IR (Loop)
TMS_sig <= '1'; wait until falling_edge(TCK_sig); -- To Exit1-IR
TMS_sig <= '0'; wait until falling_edge(TCK_sig); -- To Pause-IR
TMS_sig <= '0'; wait until falling_edge(TCK_sig); -- Stay in Pause-IR (Loop)
TMS_sig <= '1'; wait until falling_edge(TCK_sig); -- To Exit2-IR
TMS_sig <= '0'; wait until falling_edge(TCK_sig); -- Back to Shift-IR
TMS_sig <= '1'; wait until falling_edge(TCK_sig); -- To Exit1-IR
TMS_sig <= '1'; wait until falling_edge(TCK_sig); -- To Update-IR
TMS_sig <= '0'; wait until falling_edge(TCK_sig); -- Back to Run-Test/Idle
```

-- 4. Emergency Reset Control (5 consecutive '1')

```
TMS_sig <= '1'; wait until falling_edge(TCK_sig); -- To Select-DR
TMS_sig <= '1'; wait until falling_edge(TCK_sig); -- To Select-IR
TMS_sig <= '1'; wait until falling_edge(TCK_sig); -- To Test-Logic-Reset (Path 1)
TMS_sig <= '1'; wait until falling_edge(TCK_sig); -- Stay Reset (Path 2)
```

Testing and Reliability of Electronic Systems
Athanasios Fytilis 5381

```
report "Full FSM Path Coverage Finished!" severity note;  
assert false report "End" severity failure;  
end process;  
end behavior;
```